

GeoBikeLLM: LLM-Powered Bike Route Planning with Air Quality and Vegetation-Aware Geospatial Intelligence

Sara Alshamsi*, Menatalla Haggag*, Lubana Al Rayes*, Esraa Alhamid*, Isam Al Jawarneh*

*Computer Science Department, University of Sharjah, Sharjah, UAE

{ U24102808, U23103328, U23103394, U24105438, ijawarneh}@sharjah.ac.ae

Abstract— Bike riding is widely embraced as a healthy, eco-friendly mode of transportation; thus, routes with cleaner air and greener surroundings are often preferred by cyclists seeking both physical well-being and enjoyable travel experiences. However, existing navigation systems rarely consider environmental conditions such as air pollution or vegetation density. To address this, we introduce *GeoBikeLLM*, a system that leverages Large Language Models (LLMs) for environment-aware bike route planning. Focused on Chicago, Illinois, USA, the system enables users to enter natural language prompts and their routing preferences, such as shortest, healthiest (low PM2.5), or greenest (high Normalized Difference Vegetation Index (NDVI)), or a combination of these. The system integrates air quality, vegetation, and bicycle infrastructure datasets, applies KD-Tree-based spatial matching, and computes optimized paths using a weighted graph and Yen's K-Shortest Paths algorithm. Data visualizations highlight spatial patterns in pollution and greenery. Evaluation using 35 user-generated prompts showed 94.3% accuracy in extracting both location entities and route preferences. *GeoBikeLLM* demonstrates how LLMs can enhance personalized, environmentally informed mobility experiences, offering a scalable solution for sustainable urban navigation.

Keywords—Geospatial, LLM, Air Quality, NDVI, GIS

I. INTRODUCTION

Cycling has emerged as a sustainable and health-conscious mode of urban transportation, offering benefits such as reduced carbon emissions, improved physical fitness, and alleviation of traffic congestion [1]. However, traditional navigation systems tend to prioritize shortest or fastest routes, often overlooking critical environmental factors such as air quality and green space exposure, elements that significantly impact the well-being and overall experience of cyclists. In metropolitan areas like Chicago, where both pollution and vegetation levels vary spatially [2], the ability to incorporate environmental awareness into route planning can substantially enhance the quality of urban mobility.

This study introduces *GeoBikeLLM*, an intelligent, geospatially-aware routing system that leverages Large Language Models (LLMs) to offer personalized bike route planning based on user-defined environmental preferences. The system enables cyclists to interact with user-friendly interface, using natural language prompts such as "How can

I bike from Navy Pier to Hyde Park with the cleanest air?" and automatically interprets their routing intent to deliver paths that are shortest, healthiest (least polluted), greenest (most vegetated), or a weighted combination of these criteria. This is achieved by integrating three spatial datasets: PM2.5 air quality sensor data, Normalized Difference Vegetation Index (NDVI)-based vegetation indices, and OpenStreetMap-derived bicycle infrastructure. Environmental data are typically heterogeneous in nature, often differing in format, spatial resolution, and semantics [3-5].

The city of Chicago, Illinois, USA was selected as a case study for implementation due to its diverse urban topology, extensive bike network, and availability of rich environmental datasets. The system performs data exploration and visualization to highlight the spatial distribution of air pollution and green cover, and employs a multi-criteria pathfinding algorithm that incorporates environmental penalties and rewards into graph-based route computation. A user-friendly interface was developed to allow real-time adjustment of environmental preferences via sliders, thereby decoupling preference input from rigid natural language formulation.

GeoBikeLLM showcases how LLMs can facilitate intuitive interaction with complex geospatial systems and serves as a prototype for environmentally informed mobility solutions that can be adapted to other cities. By combining natural language processing, spatial analysis, and environmental awareness, this research contributes to the growing field of GeoAI and offers a scalable, human-centric approach to sustainable urban navigation.

II. RELATED WORK

Recent advancements in GeoAI and LLM research have enabled natural language interfaces for querying and interpreting geospatial data. Studies have explored both LLM-augmented spatial querying and multimodal systems integrating maps, satellite imagery, and text for environmental reasoning. This section reviews existing approaches and highlights the gap that our proposed *GeoBikeLLM* system aims to fill.

A. LLM-Augmented Geospatial Understanding and Querying

Recent research has focused on enhancing geospatial data accessibility through natural language interfaces powered by LLMs. Systems like ChatGeoAI by Mansourian and

Oucheikh [6] and the SQL-generation framework by Jiang and Yang [7] enable users to perform GIS tasks or query spatial databases using natural language. ChatGeoAI employs LLaMA-2 to convert queries into executable PyQGIS scripts, while Jiang and Yang use ChatGPT to generate PostGIS-compatible SQL. Text-to-OverpassQL by Staniek et al. [8] further advances this by translating natural language into OverpassQL queries for OpenStreetMap, democratizing access to structured geodata.

Other studies investigate how LLMs can interpret or extract geographic knowledge. GeoQAMap by Feng et al. [9] utilizes GPT-3.5 and Wikidata to answer geographic questions and visualize answers on maps. GeoAgent by Huang et al. [10] focuses on address standardization using ChatGLM-6B, showing how LLMs can resolve ambiguous and incomplete geospatial texts. Meanwhile, Bhandari et al. [11] evaluate whether LLMs are geospatially knowledgeable, revealing that while some geospatial knowledge exists in pretrained models, it is often imprecise.

To overcome these limitations, domain-adapted models have emerged. BB-GeoGPT by Zhang et al. [12] is a fine-tuned LLaMA-2 model trained on GIS texts and Q&A pairs, outperforming general LLMs in spatial queries. Similarly, GeoLLM by Shah et al. [13] boosts geospatial prediction accuracy by embedding map-based contextual data into prompts. These efforts establish the foundation for natural language interaction with spatial systems but stop short of performing complex geospatial planning or optimization.

B. Multimodal Geospatial Intelligence for Environmental and Spatial Planning

Advancements in GeoAI have expanded to multimodal systems integrating LLMs with visual and structured geospatial data. OmniGeo by Yuan et al. [14] exemplifies this trend by combining satellite imagery, map layers, and text data, enabling LLMs to perform diverse tasks such as land use classification and environmental forecasting. Geode by Wu et al. [15] and GTChain by Zhang et al. [16] extend LLMs into autonomous agents, coordinating APIs, GIS tools, and real-time databases to solve spatial tasks.

These systems demonstrate the potential of LLMs in spatial reasoning, data retrieval, and geospatial tool orchestration. Geode performs zero-shot geospatial Q&A with explicit reasoning and live data, while GTChain trains LLMs to generate tool-use chains for complex tasks in simulated GIS environments. The autonomous GIS agent by Ning et al. [17] also showcases how LLMs can dynamically retrieve data across diverse spatial datasets, enhancing spatial decision-making workflows.

Despite these advancements, existing solutions lack targeted support for personalized route planning or environmental criteria integration. None of the reviewed systems directly tackle bicycle routing that adapts to vegetation or pollution levels. Our proposed GeoBikeLLM addresses this gap by combining LLMs with geospatial intelligence to deliver environment-aware, health-optimized bike route planning.

III. METHODOLOGY

This section outlines the design and implementation of the GeoBikeLLM system, which integrates geospatial data, environmental metrics, and LLMs to provide intelligent, eco-aware bike routing in Chicago. The methodology covers the

datasets used, visualization techniques, route selection logic, and the role of LLMs in query interpretation and explanation generation. An overview of the entire system workflow is illustrated in Figure 1.

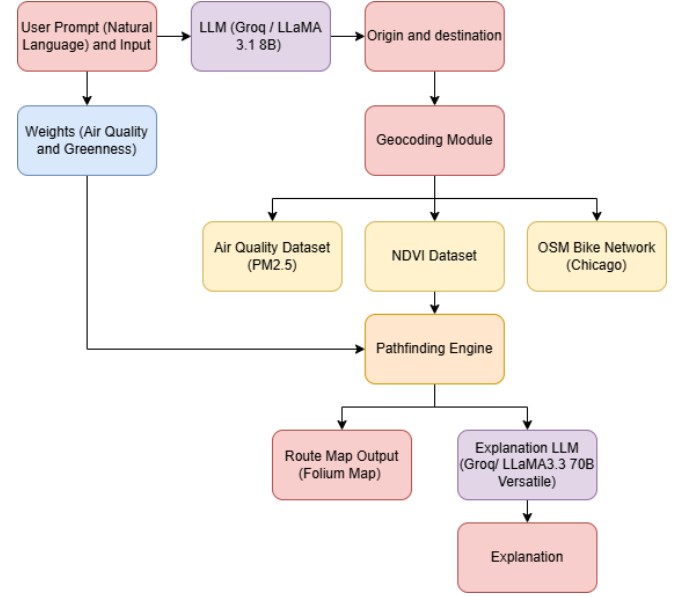


Fig. 1: Overview of the GeoBikeLLM system

A. Datasets

To support environment-aware and health-optimized route planning in Chicago, this study integrates three geospatial datasets: air quality measurements, vegetation indices, and bike infrastructure.

1) *Air Quality Data*: The air quality data is sourced from the Chicago Eclipse Air Quality Dataset, which contains sensor-based measurements of particulate matter (PM2.5) across various locations in the city. Each record includes the geographic coordinates of the sensor, the timestamp of the reading, and the corresponding PM2.5 concentration. These data help evaluate local pollution levels along potential bike routes.

2) *NDVI Dataset*: The NDVI dataset provides spatial measurements of vegetation health across Chicago. Each entry corresponds to a specific location defined by geographic coordinates and includes a ‘grid_code’ representing the NDVI value. Higher NDVI scores indicate greener areas, making this dataset essential for assessing environmental quality and greenery exposure during routing.

3) *Bicycle Network Dataset*: This dataset captures the spatial structure of the bicycle infrastructure in Chicago, derived from OpenStreetMap (OSM). It includes details on individual bike path segments, such as start and end coordinates, segment lengths, and geometry data. The dataset supports the construction of a navigable network for route computation between user-defined locations.

These datasets collectively enable the integration of air quality and environmental context into the spatial decision-making process for bicycle routing.

B. Dataset Visualization and Exploration

To better understand the spatial distribution of air pollution and vegetation in Chicago, we performed

exploratory visualizations using Folium and Python libraries. PM2.5 sensor data was plotted to identify high-pollution zones, while NDVI values were visualized to detect greener regions.

1) Air Quality Visualization

To visually explore the air quality dataset, a heatmap was generated using the PM2.5 readings across various geographic locations in Chicago. The map shown in Figure 2 represents the normalized PM2.5 values, with areas of darker intensity indicating higher pollution concentrations. The heatmap was created by aggregating data from 19 CSV files hosted on GitHub, each containing latitude, longitude, and PM2.5 values for different parts of the city. After cleaning and normalizing the data using Min-Max scaling, each point was plotted using Folium's HeatMap plugin. The visualization reveals several hotspots of elevated pollution, particularly along dense urban corridors and industrial zones. This spatial representation helps identify areas that may require further attention or intervention for environmental and public health planning.

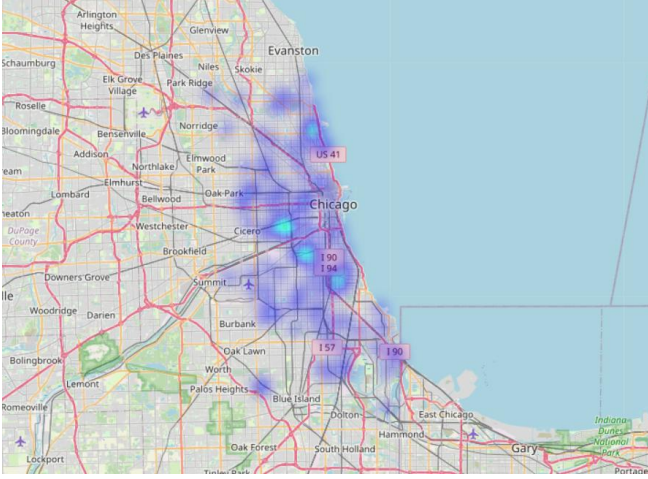


Fig. 2: Air quality visualization

2) NDVI Visualization

Figure 3 illustrates the spatial distribution of vegetation across the Chicago area using a heatmap based on normalized NDVI values. Areas with higher vegetation density are shown in green, while lower-density zones appear blue. Dense vegetation is concentrated in the western and southwestern parts of the city, corresponding to parklands and suburban zones. In contrast, central urban areas and the lakefront exhibit lower NDVI scores, indicating minimal greenery.

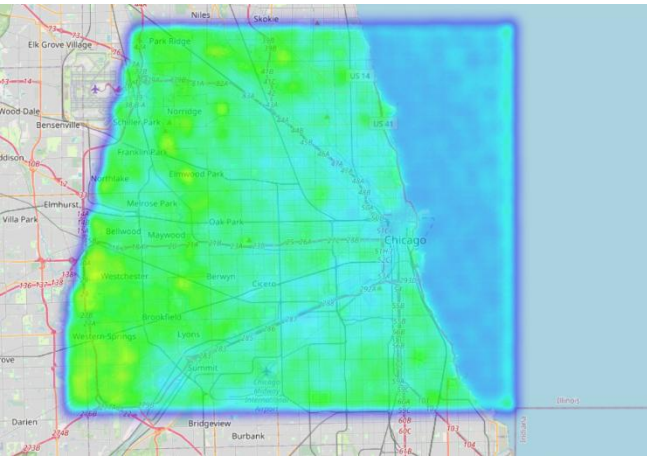


Fig. 3 NDVI visualization

southwestern parts of the city, corresponding to parklands and suburban zones. In contrast, central urban areas and the lakefront exhibit lower NDVI scores, indicating minimal greenery.

This visualization provides critical insights into urban green coverage, supporting applications in environmental health analysis, route planning, and urban sustainability strategies. The results emphasize the importance of integrating vegetation data when assessing urban livability and ecosystem services.

C. Path Selection Module

The core of the route planning logic involves constructing a weighted undirected graph from the Chicago bike lane dataset. Each edge represents a bike segment with an associated length, while nodes correspond to geospatial coordinates. To account for environmental quality, air pollution data (PM2.5) and vegetation density (NDVI) are spatially matched to each segment using nearest-neighbor querying via KD-Trees. The cost of each edge is computed as shown in (1), where α and β control the influence of PM2.5 and NDVI, respectively.

$$Cost = length \times (1 + \alpha \cdot PM2.5 - \beta \cdot NDVI) \quad (1)$$

Where α and β are tunable weights for penalizing poor air quality and rewarding greenery, respectively.

To determine optimal bike paths, the system connects the user-specified origin and destination to the nearest nodes in the network via virtual edges. Then, using Yen's algorithm, the top-k shortest paths are extracted based on the computed cost. A penalty is applied to reused edges across iterations to encourage route diversity. For each path, average PM2.5 and NDVI values are computed, and the route with the lowest pollution and highest greenery score is selected as the recommended path. This environmentally informed selection approach balances travel distance with health and ecological considerations, enabling context-aware route planning.

The final route selection prioritizes the path with the lowest average PM2.5 and highest NDVI, promoting health and greenery. Start and end points are connected to the graph using virtual edges to ensure complete coverage, and edge reuse is penalized to promote diversity among the top-k results.

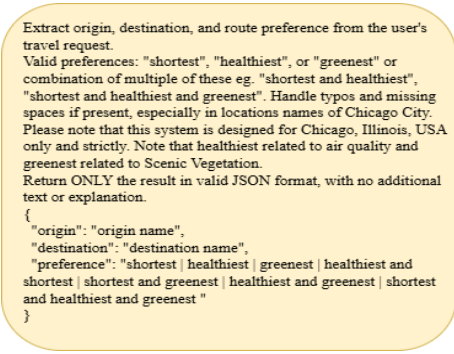
D. LLM Integration

In the proposed routing system, two LLMs are integrated using the Groq API to enhance the NLP and explanation capabilities of the application.

1) Query Interpretation using LLaMA 3.1 8B Instant:

This model is used in the `extract_trip_info(prompt)` function. It parses free-form user prompts to extract three structured elements: the trip's origin, the destination, and the routing preference (e.g., shortest, greenest, or healthiest). The system prompt explicitly instructs the model to focus on the Chicago area and to correct for common user input issues such as misspellings, missing spaces, or informal phrasing. This design ensures that users can interact with the system using natural language without adhering to strict formatting, improving accessibility and usability. As shown in Figure 4, the prompt used for the LLM clearly defines the expected

output format and constraints tailored to Chicago-specific queries.



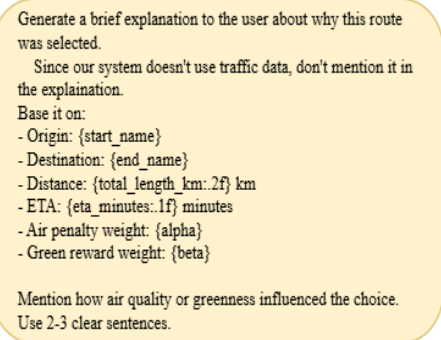
```

Extract origin, destination, and route preference from the user's travel request.
Valid preferences: "shortest", "healthiest", or "greenest" or combination of multiple of these eg. "shortest and healthiest", "shortest and healthiest and greenest". Handle typos and missing spaces if present, especially in locations names of Chicago City. Please note that this system is designed for Chicago, Illinois, USA only and strictly. Note that healthiest related to air quality and greenest related to Scenic Vegetation.
Return ONLY the result in valid JSON format, with no additional text or explanation.
{
  "origin": "origin name",
  "destination": "destination name",
  "preference": "shortest | healthiest | greenest | healthiest and shortest | shortest and greenest | healthiest and greenest | shortest and healthiest and greenest"
}

```

Fig. 4 Structured system prompt used to extract origin, destination, and routing preference from natural language input

2) *LLaMA 3.3 70B Versatile for Explanation Generation*: This larger model is employed to generate concise and informative textual explanations for the recommended route. It was chosen for its advanced reasoning capabilities and high-quality language generation, which are essential for summarizing spatial decisions involving air quality, greenness, and distance. By presenting these factors in a user-friendly format, the model enhances the system's interpretability and transparency. As shown in Figure 5, the prompt provides a structured template that guides the LLM in producing 2–3 clear sentences explaining how environmental criteria influenced the recommended route.



```

Generate a brief explanation to the user about why this route was selected.
Since our system doesn't use traffic data, don't mention it in the explanation.
Base it on:
- Origin: {start_name}
- Destination: {end_name}
- Distance: {total_length km:.2f} km
- ETA: {eta_minutes:.1f} minutes
- Air penalty weight: {alpha}
- Green reward weight: {beta}

Mention how air quality or greenness influenced the choice.
Use 2-3 clear sentences.

```

Fig. 5 Prompt used to instruct the LLM to generate concise, user-friendly explanations for the selected route

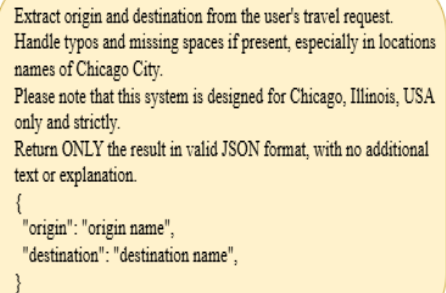
3) *Prompt Engineering*: A critical component of GeoBikeLLM's development was the iterative process of prompt engineering, which played a central role in ensuring reliable and accurate interaction between the user and the system. Prompt engineering refers to the design and refinement of natural language inputs that guide the behavior of LLMs [18]. In our implementation, we tested and optimized multiple prompt formulations to achieve consistent extraction of origin, destination, and routing preferences across varied user inputs, including those with informal phrasing or typographical errors. This highlights the importance of prompt optimization for improving system robustness and for enhancing user experience, ensuring that the LLM can interpret diverse expressions of intent with precision and reliability.

4) *LLM Evaluation*: To evaluate the effectiveness of the LLM in accurately extracting structured information from natural language prompts, we conducted a controlled test using 35 user-generated inputs. Each prompt reflected realistic and diverse travel requests, including typographical errors, informal phrasing, and mixed formatting. At the time of testing, the system was configured to extract both the origin and destination, as well as routing preferences (e.g., healthiest, shortest, greenest). The extracted values were compared against the manually annotated ground truth for each prompt. Accuracy was computed based on the correct identification of both location entities and preference tags. This evaluation helped validate the reliability of the LLM in parsing location-based instructions under noisy real-world conditions and informed our later decision to shift preference input from text to user-adjustable sliders for greater precision and control.

The integration of LLMs in this project enhances accessibility by enabling natural language interaction with complex geospatial computations. The system can interpret a wide range of user inputs, including informal phrasing, typographical errors, inconsistent formatting, and loosely defined place names. This eliminates the need for rigid query structures or rule-based parsing, allowing users to communicate using everyday language. In addition to interpreting queries, the LLMs generate clear and concise explanations for routing decisions, improving transparency and user trust.

E. Overall System

The GeoBikeLLM system enables natural language-based bike route planning within Chicago, Illinois, by integrating environmental and spatial data to recommend optimized cycling paths. Users interact with the system by entering a prompt such as *"I want to go from Melrose Park to Hyde Park."* The system employs LLM to extract only the origin and destination from the prompt, accounting for informal phrasing, typos, and spacing issues in location names. The system prompt used to guide this extraction is shown in Figure 6, which explicitly restricts the task to valid JSON output and enforces geographic constraints within Chicago.



```

Extract origin and destination from the user's travel request.
Handle typos and missing spaces if present, especially in locations names of Chicago City.
Please note that this system is designed for Chicago, Illinois, USA only and strictly.
Return ONLY the result in valid JSON format, with no additional text or explanation.
{
  "origin": "origin name",
  "destination": "destination name",
}

```

Fig. 6 System prompt provided to the LLM for extracting origin and destination

To offer users greater control over route optimization, the system provides two adjustable sliders in the user interface: one for Air Quality Penalty and another for Greenness Reward. These sliders eliminate the need to express routing preferences within the prompt itself. The air quality slider (α)

penalizes routes with higher PM2.5 levels, while the greenness slider (β) favors paths with higher NDVI values.

Once the origin and destination are identified, the system constructs a geospatial graph of Chicago's bike network. Each edge in the graph is dynamically weighted based on nearby air quality and vegetation data obtained from environmental sensors. Route computation is carried out using an adapted Yen's K-Shortest Paths algorithm, which generates multiple path options while incorporating the user-defined environmental weights.

Finally, the system displays an interactive route map, computes the Estimated Time of Arrival (ETA), and uses a second LLM query to generate a concise natural language explanation of the recommended route. This explanation reflects how the selected route aligns with the user's environmental preferences.

IV. RESULTS

This section outlines the results of testing the system, divided into three parts.

A. Best Path Selection (Phase 1)

The best path selection testing evaluates the system's ability to recommend optimal biking routes based on different user-defined criteria such as distance, air quality (PM2.5), and vegetation coverage (NDVI).

A simple user input function was implemented to prompt users to enter a starting location, destination, and select one of three ranking criteria:

1. Greenest Path (based on NDVI)
2. Least Polluted Path (based on PM2.5)
3. Shortest Path

The system then geocodes the input locations, computes multiple possible routes using real geospatial data, and ranks them based on the selected criterion. Below are three examples from the testing phase:

```
Enter the start location (e.g., Melrose Park): Melrose Park
Enter the end location (e.g., Hyde Park): Hyde Park
WARNING:urllib3.connectionpool:Retrying (Retry(total=1, connect=None,
Start coordinates: (-87.6796849, 41.9404726)
End coordinates: (-87.5939244, 41.7944464)

Choose the ranking criteria:
1. Shortest path
2. Least polluted path (lowest PM2.5)
3. Greenest path (highest NDVI)

Enter the number corresponding to your choice (1, 2, or 3): 3

Ranking based on: Greenest Path
Best: 22.32 km, Avg PM2.5: 4.51, Avg NDVI: 0.13
Path 2: 20.57 km, Avg PM2.5: 5.19, Avg NDVI: 0.13
Path 3: 22.76 km, Avg PM2.5: 5.06, Avg NDVI: 0.12
Path 4: 22.35 km, Avg PM2.5: 4.81, Avg NDVI: 0.12
Path 5: 22.75 km, Avg PM2.5: 5.09, Avg NDVI: 0.10
```

Fig. 7 Console output for greenest path

Example 1

Start location: Melrose Park

Destination: Hyde Park

Criteria: 3. Greenest Path (highest NDVI)

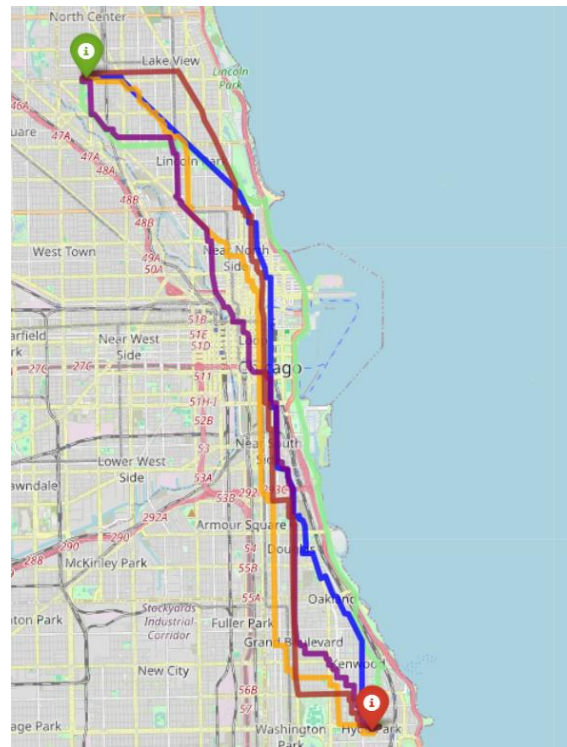


Fig. 8 Route map visualization for greenest path

The system was able to successfully retrieve the geographic coordinates for both locations and presented a menu to choose the ranking criteria: shortest path, least polluted path (PM2.5), or greenest path (NDVI). The user chose to prioritize the greenest path.

Based on this, the system evaluated multiple possible routes and selected the one with the highest average NDVI (0.13), indicating better vegetation and greener surroundings. This best route is 22.32 km long with an average PM2.5 of 4.51. Other possible routes are listed with their distances, pollution levels, and greenery scores, but the top-ranked one is considered the greenest (highest average NDVI).

Figure 7 presents a snapshot of the output generated when executing the code. Figure 8 illustrates a map displaying the five candidate routes, where the best path is drawn in green. This visual shows a clear comparison of route paths based on the selected criteria.

```
Enter the start location (e.g., Melrose Park): Navy Pier
Enter the end location (e.g., Hyde Park): Millennium Park
Start coordinates: (-87.6041447, 41.8917998)
End coordinates: (-87.6225361, 41.8825754)

Choose the ranking criteria:
1. Shortest path
2. Least polluted path (lowest PM2.5)
3. Greenest path (highest NDVI)

Enter the number corresponding to your choice (1, 2, or 3): 2

Ranking based on: Least Polluted Path
Best: 2.66 km, Avg PM2.5: 3.05, Avg NDVI: 0.11
Path 2: 2.45 km, Avg PM2.5: 3.07, Avg NDVI: 0.09
Path 3: 2.27 km, Avg PM2.5: 3.10, Avg NDVI: 0.08
Path 4: 2.31 km, Avg PM2.5: 3.10, Avg NDVI: 0.08
Path 5: 3.19 km, Avg PM2.5: 3.20, Avg NDVI: 0.06
```

Fig. 9 Console output for least polluted path

Example 2

Start location: Navy Pier

Destination: Millennium Park

Criteria: 2. Least Polluted Path (lowest PM2.5)

The system successfully found five biking paths between Navy Pier and Millennium Park in Chicago, ranking them based on air quality (PM2.5 levels).

The best route is 2.66 kilometers long with the lowest average PM2.5 value of 3.05, which makes it the least polluted route. This path also has the highest NDVI (0.11), indicating more greenery compared to the other routes.

Although some alternative paths were shorter, the best route was selected based on the criterion of lowest air pollution. The system used Yen's K-shortest paths method to evaluate multiple route options and chose the least polluted and greenest option as the top route. The results are presented in Figures 9 and 10.

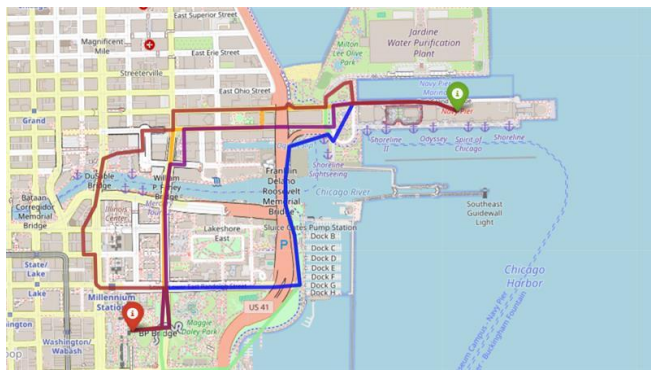


Fig. 10 Route map visualization for least polluted path

```
Enter the start location (e.g., Melrose Park): Bridgeport
Enter the end location (e.g., Hyde Park): Museum Campus
Start coordinates: (-87.6493815, 41.8363967)
End coordinates: (-87.6156365, 41.8685933)

Choose the ranking criteria:
1. Shortest path
2. Least polluted path (lowest PM2.5)
3. Greenest path (highest NDVI)

Enter the number corresponding to your choice (1, 2, or 3): 1

Ranking based on: Shortest Path
Best: 5.85 km, Avg PM2.5: 5.29, Avg NDVI: 0.08
Path 2: 6.53 km, Avg PM2.5: 5.27, Avg NDVI: 0.08
Path 3: 7.74 km, Avg PM2.5: 6.52, Avg NDVI: 0.11
Path 4: 7.84 km, Avg PM2.5: 6.21, Avg NDVI: 0.12
Path 5: 9.36 km, Avg PM2.5: 4.94, Avg NDVI: 0.08
```

Fig. 11 Console output for shortest distance

Example 3

Start location: Bridgeport

Destination: Museum Campus

Criteria: Shortest Distance

In this example, the user chose to search for the shortest path between Bridgeport and Museum Campus. The system successfully computed five candidate routes and ranked them by distance.

The top-ranked route is 5.85 km long, with an average PM2.5 of 5.29 and an average NDVI of 0.08. Figure 11 displays the console output summarizing the route details and rankings.

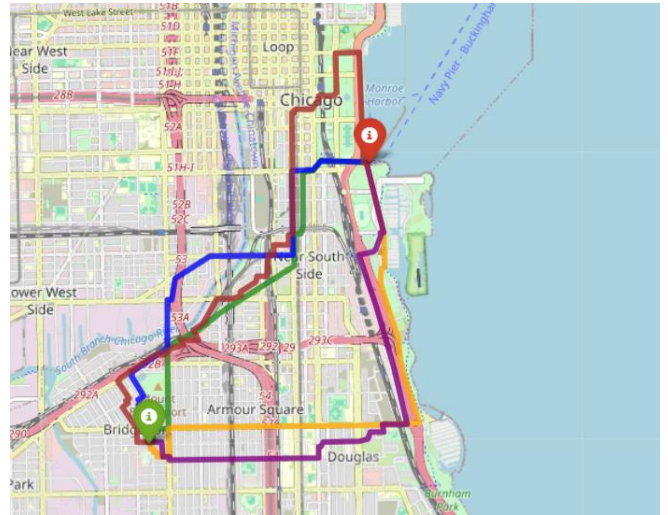


Fig. 12 Route map visualization for shortest distance

Figure 12 illustrates the corresponding map with all five candidate routes, and the shortest path is highlighted in green.

To validate the results, Google Maps was used to find the distance between Bridgeport and Museum Campus, and compare the distance with what the system displayed. The system displayed a distance of 5.85 km, while Google Maps showed a distance of approximately 3.6 miles (equivalent to 5.79 km). These numbers show that the system is functioning correctly.

B. LLM Results

This subsection presents the evaluation results of the two integrated language models in GeoBikeLLM. The first model focuses on extracting structured trip details from natural language input, while the second generates explanatory feedback based on computed routes and user-defined preferences.

1) *Query Interpretation using LLaMA 3.1 8B Instant:* To assess the performance of the LLaMA 3.1 8B Instant model in query interpretation, we evaluated its ability to extract structured information from 35 user prompts. These prompts were designed to reflect diverse natural language variations, including typos, spacing issues, and informal phrasing. Out of the 35 prompts, the model correctly identified both the origin and destination in 33 cases, achieving a location extraction accuracy of 94.3%. Similarly, the model correctly extracted the user's routing preference (e.g., healthiest, shortest, greenest) in 33 out of 35 cases, also yielding a preference extraction accuracy of 94.3%. These results demonstrate the model's robustness in understanding noisy and informal user inputs. However, minor inconsistencies were observed in cases involving ambiguous phrasing or rare place names, which supports the decision to shift preference input to sliders in the final system design.

2) *LLaMA 3.3 70B Versatile for Explanation Generation:* During testing, the model consistently produced clear and contextually relevant justifications, successfully incorporating factors such as air quality and vegetation coverage into its responses. Across various prompt inputs and slider configurations, the generated explanations accurately reflected the impact of the air quality penalty (α) and

greenness reward (β) on the chosen route. For example, when higher greenness weights were applied, the model emphasized scenic, vegetation-rich paths. In contrast, when air quality was prioritized, the output referenced routes with cleaner air. The responses remained coherent, grammatically sound, and aligned with the environmental context provided. This consistency demonstrates the model's effectiveness in delivering human-readable, informative feedback, reinforcing trust in the system's recommendations.

C. Interface Prompt Testing

In this part of the project, different cycling routes were generated from Melrose Park to Hyde Park by adjusting how much importance was given to clean air and green areas. Each route reflects a different balance between travel time, distance, air quality, and greenness.

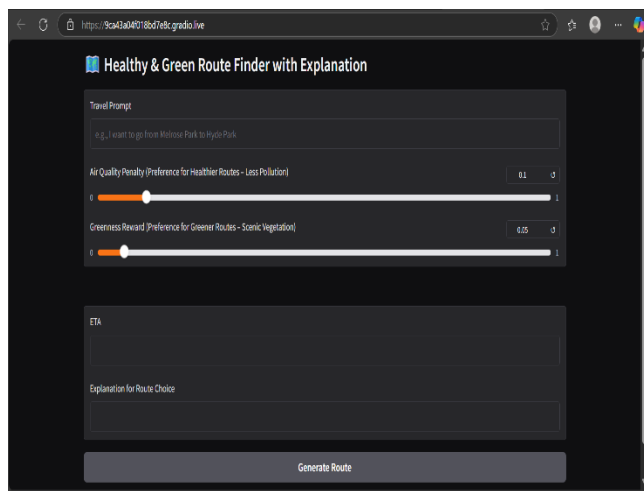


Fig. 13 System interface

Figure 13 is a snapshot of the LLM interface. The user will input the prompt in the first field, then adjust the markers to the desired weight of each aspect of the route (higher alpha for cleaner air, higher beta for greener path). The user must then click on “Generate Route” to see the results. The 'Estimated Time' field displays the time required to travel from the start to the end location, assuming a bicycle speed of 15 km/h. The “Explanation” field shows an explanation for the selected route, detailing how the assigned weights influenced the system's decision.

Route 1

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 0.25 **Greenness Reward:** 0.75

Output Route: Figure 14

Estimated Time: 139.9 minutes **Distance:** 34.99

Explanation: The selected route from Melrose Park to Hyde Park prioritizes a more environmentally friendly path, taking into account the air quality and greenness of the areas. With a higher weight given to green rewards (0.75) compared to air penalties (0.25), the route favors areas with more green spaces, resulting in a slightly longer journey of 34.99 km and an estimated 139.9 minutes. This choice reflects a consideration for a more sustainable and pleasant route, balancing the need for efficient travel with the benefits of a greener environment.

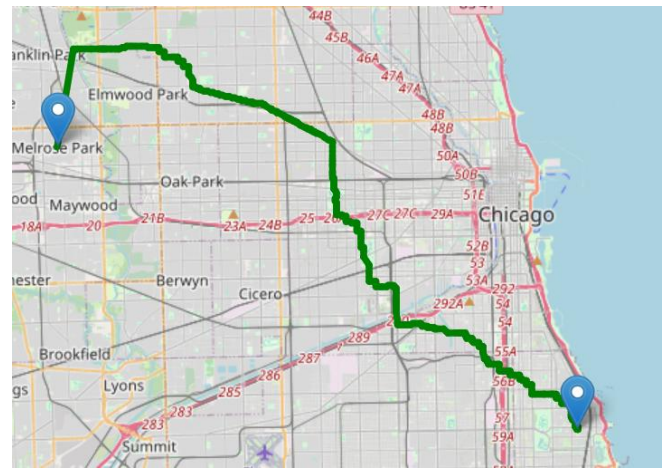


Fig. 14 Route 1 output

Route 2

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 0.1 **Greenness Reward:** 0.05

Output Route: Figure 15

Estimated Time: 133.4 minutes **Distance:** 33.34 km

Explanation: This route from Melrose Park to Hyde Park was selected as it balances distance and estimated travel time, spanning 33.34 km with an estimated 133.4 minutes of travel. The route's selection was influenced by factors such as air quality, with areas of poorer air quality being avoided due to a slight penalty, and the presence of green spaces, which were favored due to their positive impact. Overall, the chosen route optimizes your journey while considering the environmental aspects of air quality and greenness.

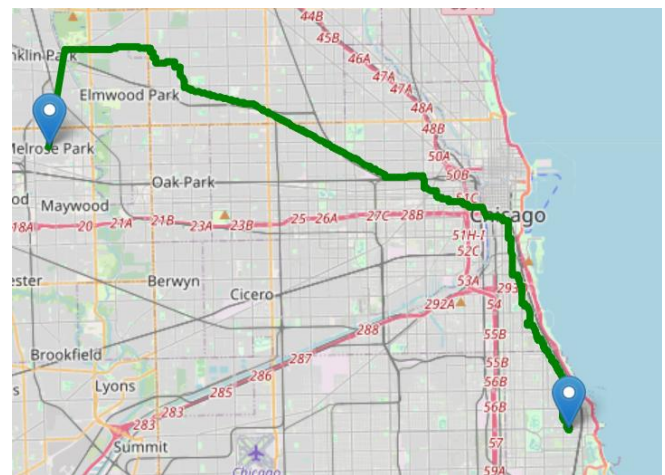


Fig. 15 Route 2 output

Route 3

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 0 **Greenness Reward:** 0

Output Route: Figure 16

Estimated Time: 131.1 minutes **Distance:** 32.78 km

Explanation: The selected route from Melrose Park to Hyde Park is approximately 32.78 km and is estimated to take around 131.1 minutes. This route was chosen as it provides a reasonable balance of distance and travel time.

Neither air quality nor greenness had a significant influence on the route selection, as the air penalty and green reward weights were set to 0, indicating that these factors did not impact the routing decision.

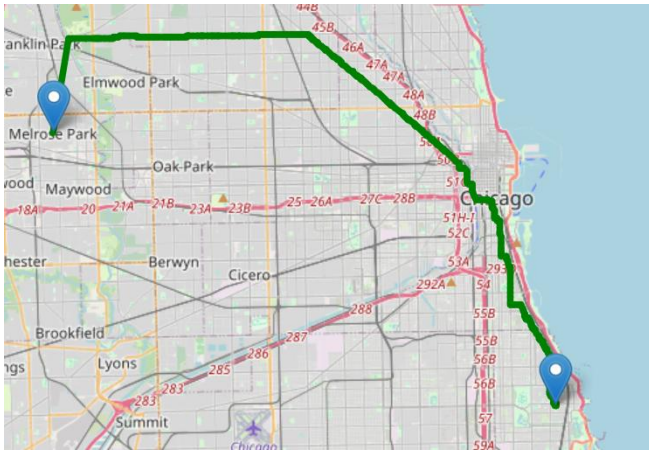


Fig. 16 Route 3 output

Route 4

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 1 **Greenness Reward:** 1

Output Route: Figure 17

Estimated Time: 139.9 minutes **Distance:** 34.98 km

Explanation: This route from Melrose Park to Hyde Park was selected as it balances distance and travel time, with an estimated 34.98 km journey taking approximately 139.9 minutes. The route's selection was also influenced by considerations of air quality and greenness, with equal weight given to minimizing air pollution and maximizing exposure to green spaces. As a result, the chosen route aims to provide a relatively balanced and environmentally conscious path to your destination.

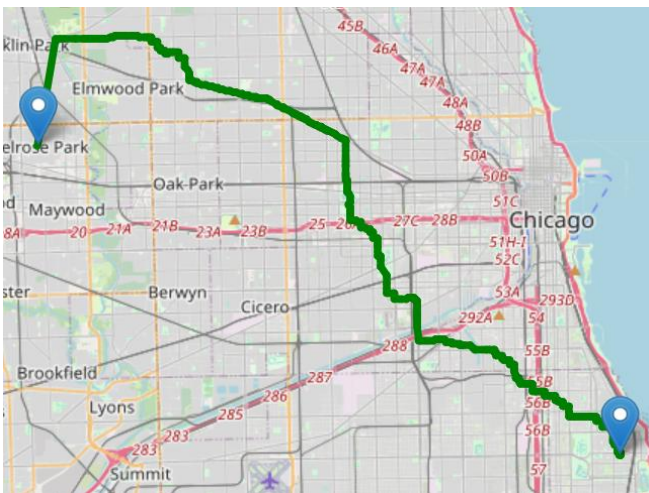


Fig. 17 Route 4 output

Route 5

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 0 **Greenness Reward:** 1

Output Route: Figure 18

Estimated Time: 134.7 minutes **Distance:** 33.65 km

Explanation: This route from Melrose Park to Hyde Park was selected as it offers a balance of distance and estimated travel time, spanning 33.67 km with an estimated time of arrival in 134.7 minutes. The route's selection was influenced by the consideration of green spaces, as the system prioritizes routes with higher greenness due to the assigned green reward weight. As a result, the chosen route likely passes through areas with more parks or green areas, contributing to a more pleasant journey.

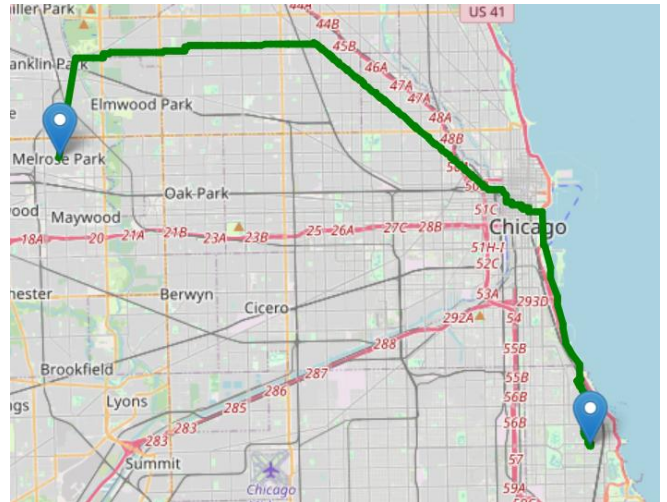


Fig. 18 Route 5 output

Route 6

Prompt: Show me a cycling route from Melrose Park to Hyde Park

Air Quality Penalty: 1 **Greenness Reward:** 0

Output Route: Figure 19

Estimated Time: 139.7 minutes **Distance:** 34.94 km

Explanation: The selected route from Melrose Park to Hyde Park, spanning 34.94 km, is estimated to take approximately 139.7 minutes. This route was chosen as it minimizes exposure to poor air quality areas, given the air penalty weight of 1, which prioritizes avoiding polluted zones. Although green spaces were not a significant factor in this selection due to a green reward weight of 0, the route still aims to provide a relatively efficient journey to your destination.

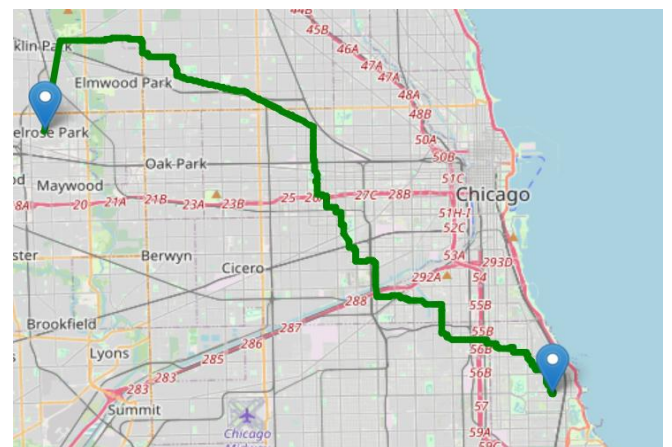


Fig. 19 Route 6 output

Route 7 (With spelling mistake)

Prompt: Show me a cycling route from Melros park to Hyde paek

Air Quality Penalty: 0 **Greenness Reward:** 1

Output Route: Figure 20

Estimated Time: 134.7 minutes **Distance:** 33.67 km

Explanation: The selected route from Melrose Park to Hyde Park, spanning 33.67 km, was chosen to optimize your journey while considering environmental factors. With a green reward weight of 1, the system prioritized a route that passes through greener areas, promoting an eco-friendlier path. As a result, the chosen route aims to balance distance and travel time with exposure to more natural environments, enhancing your overall travel experience.

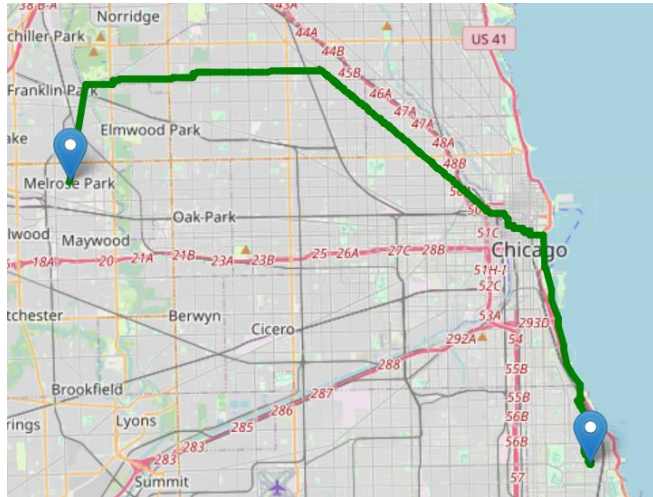


Fig. 20 Route 7 output

The results of interface testing show how the output changes based on the given weights. The same prompt was evaluated using multiple weight combinations to assess the system's performance. Additionally, the interface was tested with a prompt containing a spelling error to evaluate its robustness in handling input inaccuracies. The system performed well in calculating the path based on the chosen criteria in all cases. To validate the accuracy of the computed results, Google Maps was used as a reference for cross-verification.

V. CONCLUSION AND FUTURE WORK

GeoBikeLLM presents an innovative and practical integration of LLMs with geospatial intelligence for sustainable urban mobility. By enabling cyclists to specify their routing preferences, whether shortest, healthiest (based on PM2.5), greenest (based on NDVI), or a combination thereof, via natural language, the system offers an accessible, user-friendly interface for environment-aware bike route planning. Implemented and evaluated in Chicago, Illinois, GeoBikeLLM demonstrates how data-driven urban navigation can go beyond traditional shortest-path algorithms to prioritize environmental health and well-being. Through spatial data exploration, multimodal visualization, and robust LLM-based query interpretation and explanation, the system enhances personalized decision-making and sets a precedent for scalable, human-centric navigation solutions adaptable to other cities. This work underscores the potential of LLMs to

bridge the gap between complex geospatial data and real-world user needs, fostering more sustainable and informed urban transportation choices.

Future work will focus on extending GeoBikeLLM to support additional cities with varying environmental and infrastructure characteristics. Enhancements may include real-time pollution updates, integration of elevation data for route difficulty, and further refinement of LLM prompts.

REFERENCES

- [1] I. M. Al Jawarneh, P. Bellavista, A. Corradi, L. Foschini, and R. Montanari, "Efficiently integrating mobility and environment data for climate change analytics," in 2021 IEEE 26th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD). IEEE, 2021, pp. 1–5.
- [2] S. Azizi and T. Azizi, "Urban climate dynamics: Analyzing the impact of green cover and air pollution on land surface temperature—a comparative study across Chicago, San Francisco, and Phoenix, USA," *Atmosphere*, vol. 15, no. 8, p. 917, 2024.
- [3] I. M. Al Jawarneh, R. Montanari, and A. Corradi, "Cost-effective approximate aggregation queries on geospatial big data," in 2023 IEEE Globecom Workshops (GC Wkshps). IEEE, 2023, pp. 1313–1318.
- [4] I. M. Al Jawarneh, L. Foschini, and P. Bellavista, "Efficient integration of heterogeneous mobility-pollution big data for joint analytics at scale with QoS guarantees," *Future Internet*, vol. 15, no. 8, p. 263, 2023.
- [5] I. M. Al Jawarneh, P. Bellavista, A. Corradi, L. Foschini, and R. Montanari, "QoS-aware approximate query processing for smart cities spatial data streams," *Sensors*, vol. 21, no. 12, p. 4160, 2021.
- [1] Mansourian, A. & Oucheikh, A. (2024). ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models. *ISPRS International Journal of Geo-Information*.
- [2] Jiang, Y. & Yang, C. (2024). Is ChatGPT a Good Geospatial Data Analyst? Exploring the Integration of Natural Language into Structured Query Language within a Spatial Database. *ISPRS International Journal of Geo-Information*.
- [3] Staniek, M., et al. (2024). Text-to-OverpassQL: A Natural Language Interface for Complex Geodata Querying of OpenStreetMap. *Transactions of the Association for Computational Linguistics*.
- [4] Feng, Z., et al. (2023). GeoQAMap: Geographic Question Answering with Maps Leveraging LLM and Open Knowledge Base. *GIScience 2023, Leibniz International Proceedings in Informatics*.
- [5] Huang, W., et al. (2024). GeoAgent: To Empower LLMs using Geospatial Tools for Address Standardization. *Findings of ACL 2024*.
- [6] Bhandari, S., et al. (2023). Are Large Language Models Geospatially Knowledgeable?. *ACM SIGSPATIAL 2023*.
- [7] Zhang, Z., et al. (2024a). BB-GeoGPT: A Framework for Learning a Large Language Model for Geographic Information Science. *Information Processing & Management*.
- [8] Shah, R., et al. (2024). GeoLLM: Extracting Geospatial Knowledge from Large Language Models. *ICLR 2024 Conference Track*.
- [9] Yuan, Q., et al. (2025). OmniGeo: Towards a Multimodal Large Language Models for Geospatial Artificial Intelligence. *arXiv preprint*.
- [10] Wu, K., et al. (2024). Geode: A Zero-shot Geospatial Question-Answering Agent with Explicit Reasoning and Precise Spatio-Temporal Retrieval. *arXiv preprint*.
- [11] Zhang, Z., et al. (2025b). Geospatial Large Language Model Trained with a Simulated Environment for Generating Tool-Use Chains Autonomously. *Future Generation Computer Systems*.
- [12] Ning, Y., et al. (2025). An Autonomous GIS Agent Framework for Geospatial Data Retrieval. *International Journal of Digital Earth*.
- [13] P. Bansal, "Prompt engineering importance and applicability with generative ai," *Journal of Computer and Communications*, vol. 12, no. 10, pp. 14–23, 2024.
- [14] W. McKinney, *pandas: Powerful Python data analysis toolkit*. [Online]. Available: <https://pandas.pydata.org/>
- [15] K. Reitz, *requests: HTTP for humans*. [Online]. Available: <https://docs.python-requests.org/>

- [16] Python Software Foundation, *io* — Core tools for working with streams. [Online]. Available: <https://docs.python.org/3/library/io.html>
- [17] A. A. Hagberg, D. A. Schult, and P. J. Swart, “NetworkX: Python software for the analysis of complex networks,” in *Proc. 7th Python in Science Conf. (SciPy)*, Pasadena, CA, USA, 2008. [Online]. Available: <https://networkx.org/>
- [18] P. Bansal, “Prompt engineering importance and applicability with generative ai,” *Journal of Computer and Communications*, vol. 12, no. 10, pp. 14–23, 2024
- [19] SciPy Developers, *scipy.spatial.cKDTree* — SciPy v1.11.1 manual. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html>
- [20] Python Software Foundation, *math* — Mathematical functions. [Online]. Available: <https://docs.python.org/3/library/math.html>
- [21] Python Software Foundation, *os* — Miscellaneous operating system interfaces. [Online]. Available: <https://docs.python.org/3/library/os.html>
- [22] Python Software Foundation, *folium*: Python data. Leaflet.js maps. [Online]. Available: <https://python-visualization.github.io/folium/>
- [23] GeoPy Developers, *geopy*: Geocoding library for Python. [Online]. Available: <https://geopy.readthedocs.io/>